

# Exploration of Generative Adversarial Networks models

**Anastasiya Liavonava**

ANASTASIYA.LIAVONAVA@STUD.MIF.VU.LT

**Povilas Kvedaras**

POVILAS.KVEDARAS@STUD.MIF.VU.LT

*Deep Learning*

*Modelling and Data Analysis*

*Faculty of Mathematics and Informatics*

**Advisor:** asist. Dr. Linas Petkevičius

## Contents

|          |                                        |          |
|----------|----------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                    | <b>1</b> |
| <b>2</b> | <b>Related work</b>                    | <b>2</b> |
| <b>3</b> | <b>Data-set and data preprocessing</b> | <b>2</b> |
| <b>4</b> | <b>Models</b>                          | <b>3</b> |
| 4.1      | DCGAN . . . . .                        | 3        |
| 4.2      | CycleGAN . . . . .                     | 4        |
| <b>5</b> | <b>Results and conclusion</b>          | <b>5</b> |
| <b>6</b> | <b>Future directions</b>               | <b>8</b> |

## 1. Introduction

Generative Adversarial Networks, or GANs for short, are an approach to generative modeling using deep learning methods, such as convolutional neural networks. GANs are a clever way of training a generative model by framing the problem as a supervised learning problem with two sub-models: the generator model that we train to generate new examples, and the discriminator model that tries to classify examples as either real (from the domain) or fake (generated). The two models are trained together in a zero-sum game, adversarial, until the discriminator model is fooled about half the time, meaning the generator model is generating plausible examples.

Most popular tasks where GANs applied are: generate specific-style images or realistic photographs, image-to-image translation and text-to-image translation. In this project we aimed to perform two types of GANs (classical DCGAN and state-of-the-art cycle GAN) for two types of most spread tasks (image generation and style transferring) on Ukiyoe data set.

Technical report consist of following parts: first, we will review papers in this area, then we will present our data set, applied deep-learning architecture and finish with results and conclusions.

## 2. Related work

The GAN architecture was first described in the 2014 paper by Ian Goodfellow, et al. titled “Generative Adversarial Networks.” [1].

A standardized approach called Deep Convolutional Generative Adversarial Networks, or DCGAN, that led to more stable models was later formalized by Alec Radford, et al. in the 2015 paper titled “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks“ [4]. In our project we demonstrate image generation of artistic painting style. According to NIPS 2016 Tutorial, most GANs today are at least loosely based on the DCGAN architecture [2].

Jun-Yan Zhu in their 2017 paper titled “Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks” introduce their famous CycleGAN and a suite of very impressive image-to-image translation examples [5]. In our project we demonstrate translation from photograph to artistic painting style.

## 3. Data-set and data preprocessing

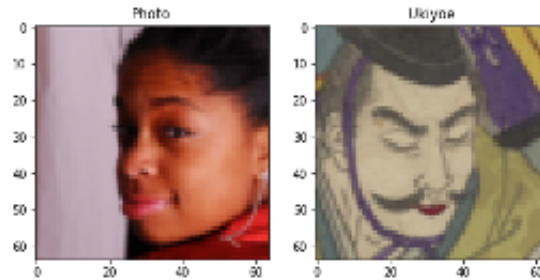
We use a data set of Ukiyo-e face images [3] for training two GAN models. The ukiyo-e faces data set comprises of 5209 images of faces from ukiyo-e prints. The images are 1024x1024 pixels in jpeg format. Below is a map of some images in the data set.

Figure 1: Examples of Ukiyo-e face images



Before running the models, we apply data preprocessing to the ukiyo-e faces data set. For this we create classes, which resize, crop, convert to tensor and normalize images. Image sizes for training –  $3 \times 64 \times 64$ . Also we apply same preprocessing to human faces data set, which we use for style transferring task. Random images from both data sets shown below.

Figure 2: Examples of preprocessed images



## 4. Models

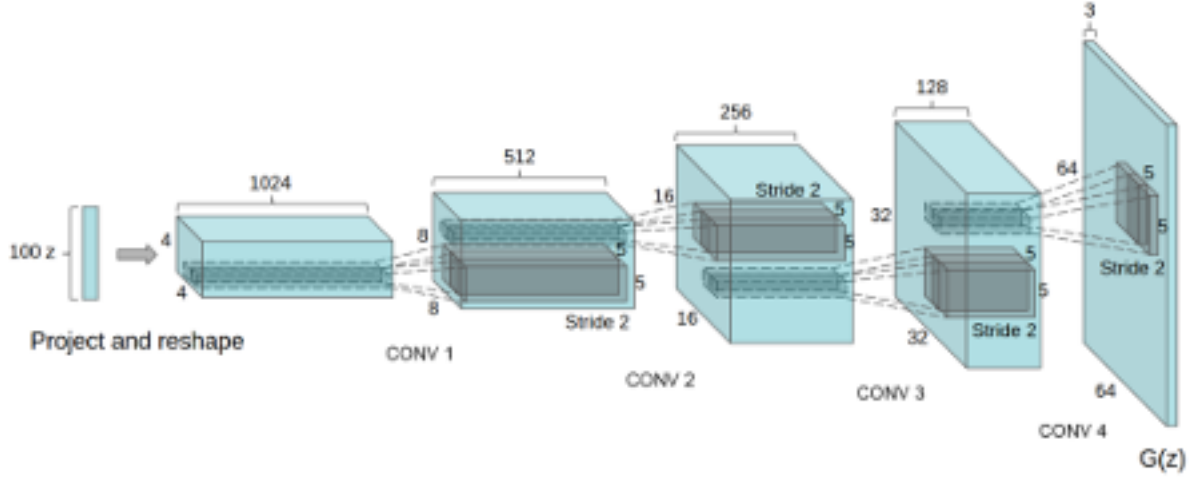
### 4.1 DCGAN

A DCGAN is a direct extension of the basic GAN, except that it explicitly uses convolutional and convolutional-transpose layers in the discriminator and generator, respectively.

The discriminator is made up of strided convolution layers, batch norm layers, and LeakyReLU activations. The input is a  $3 \times 64 \times 64$  input image and the output is a scalar probability that the input is from the real data distribution. The generator is comprised of convolutional-transpose layers, batch norm layers, and ReLU activations. The input is a latent vector,  $z$ , that is drawn from a standard normal distribution and the output is a  $3 \times 64 \times 64$  RGB image. The strided conv-transpose layers allow the latent vector to be transformed into a volume with the same shape as an image.

In the paper, the authors also give some tips about how to setup the optimizers, how to calculate the loss functions, and how to initialize the model weights. For example, from the DCGAN paper, the authors specify that all model weights shall be randomly initialized from a normal distribution with  $\text{mean}=0$ ,  $\text{stdev}=0.02$ .

Figure 3: Image from the paper. DCGAN generator. A 100 dimensional uniform distribution  $Z$  is projected to a small spatial extent convolutional representation with many feature maps. A series of four fractionally-strided convolutions then convert this high level representation into a  $64 \times 64$  pixel image. Notably, now fully connected or pooling layers are used



## 4.2 CycleGAN

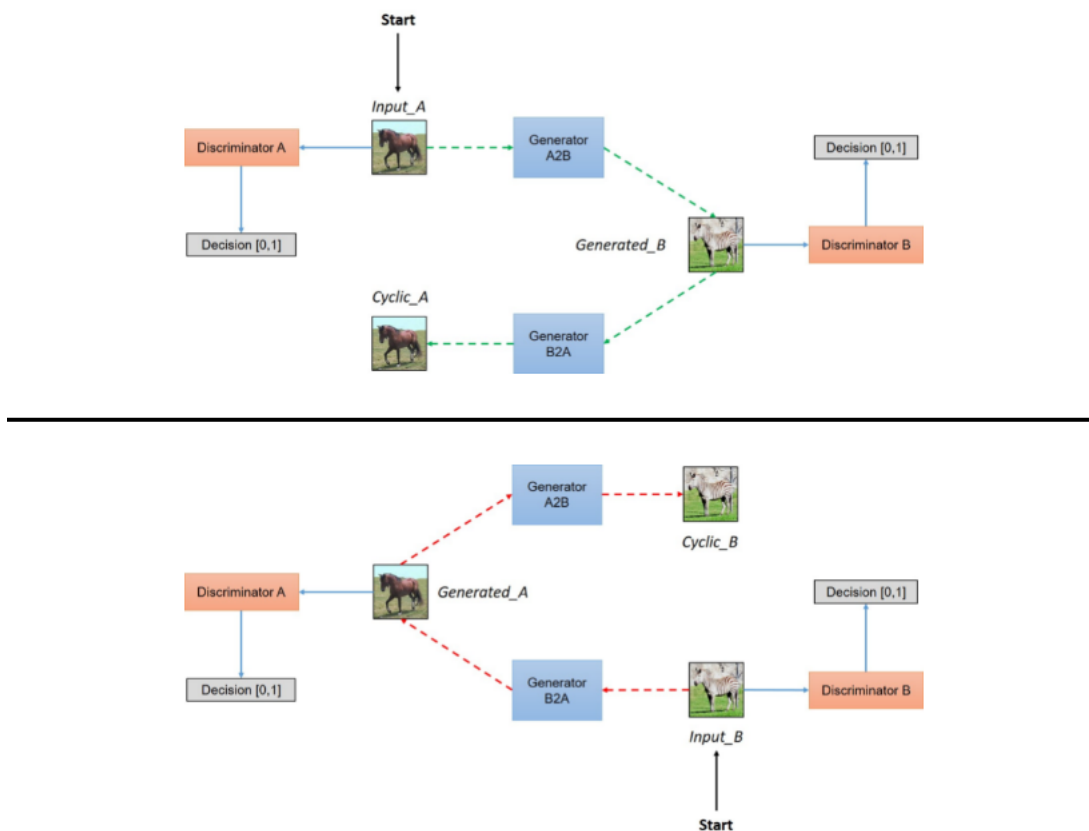
CycleGAN, is an approach to training a deep convolutional neural network for image-to-image translation tasks. Unlike other GAN models for image translation, the CycleGAN does not require a data set of paired images.

The model architecture is comprised of two generator models: one generator (Generator-A) for generating images for the first domain (Domain-A) and the second generator (Generator-B) for generating images for the second domain (Domain-B). The generator models perform image translation, meaning that the image generation process is conditional on an input image, specifically an image from the other domain. Generator-A takes an image from Domain-B as input and Generator-B takes an image from Domain-A as input.

Each generator has a corresponding discriminator model. The first discriminator model (Discriminator-A) takes real images from Domain-A and generated images from Generator-A and predicts whether they are real or fake. The second discriminator model (Discriminator-B) takes real images from Domain-B and generated images from Generator-B and predicts whether they are real or fake.

The discriminator and generator models are trained in an adversarial zero-sum process, like in normal GAN models. The generators learn to better fool the discriminators and the discriminator learn to better detect fake images. Together, this model finds an equilibrium during the training process.

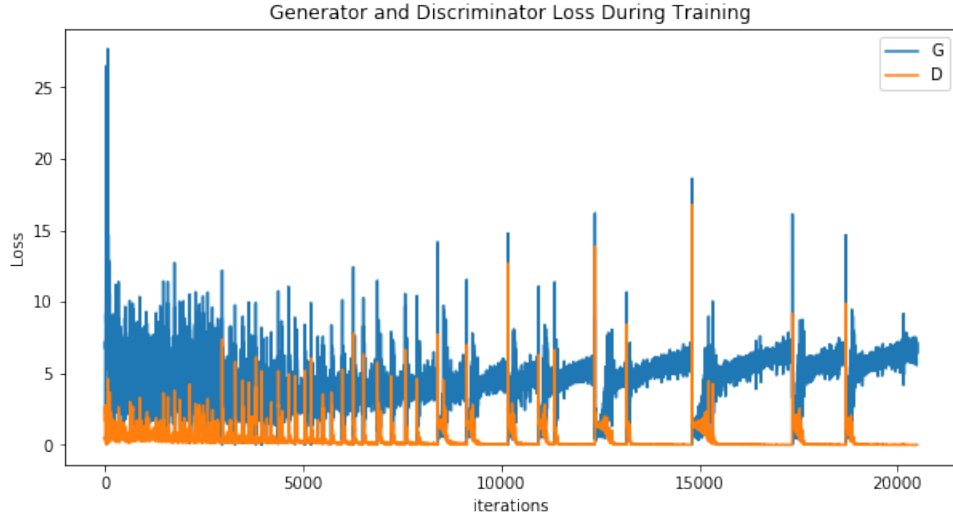
Figure 4: A simplified architecture of Cycle-GAN (Source: Hardik Bansal on GAN)



## 5. Results and conclusion

First, we train the model with 500 epochs. Loss during training is shown below.

Figure 5: DCGAN Generator and Discriminator Loss During Training (by each 10 batch on each epoch)



After training we got following output.

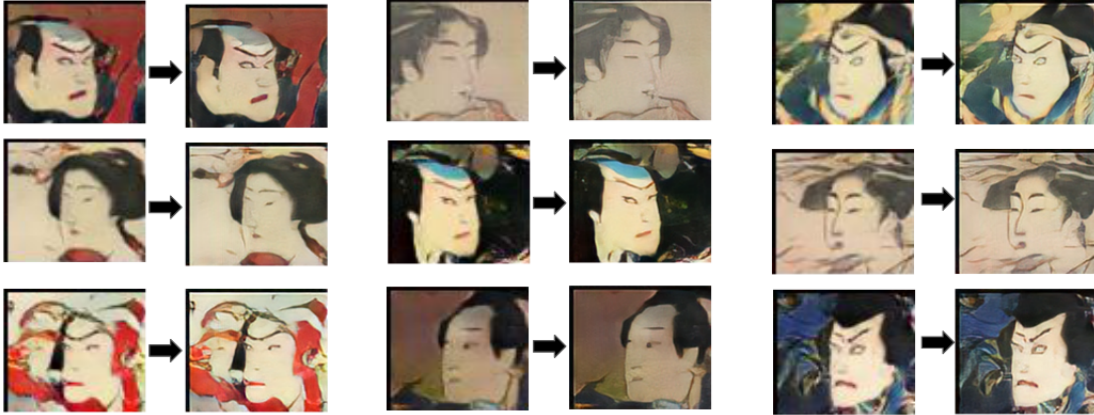
Figure 6: Generated images



Also, as image size is quite small, pretrained image super resolution model is used to make the result of better quality (<https://github.com/idealo/image-super-resolution>). Ex-

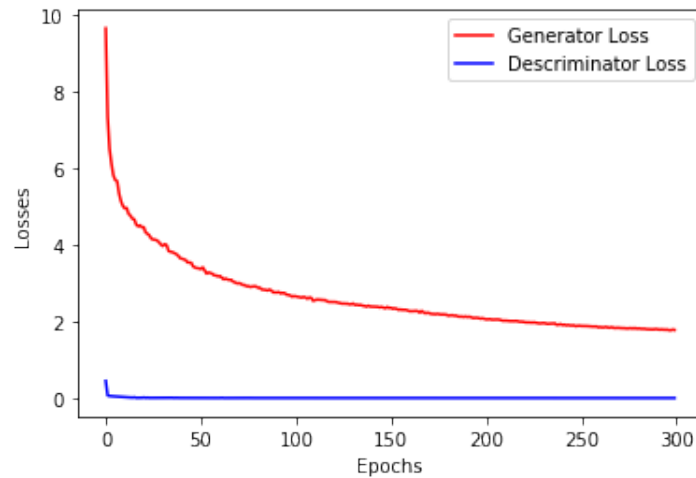
amples of implementing Residual Dense Network for Image Super-Resolution is shown in the figure 7.

Figure 7: Examples of final result



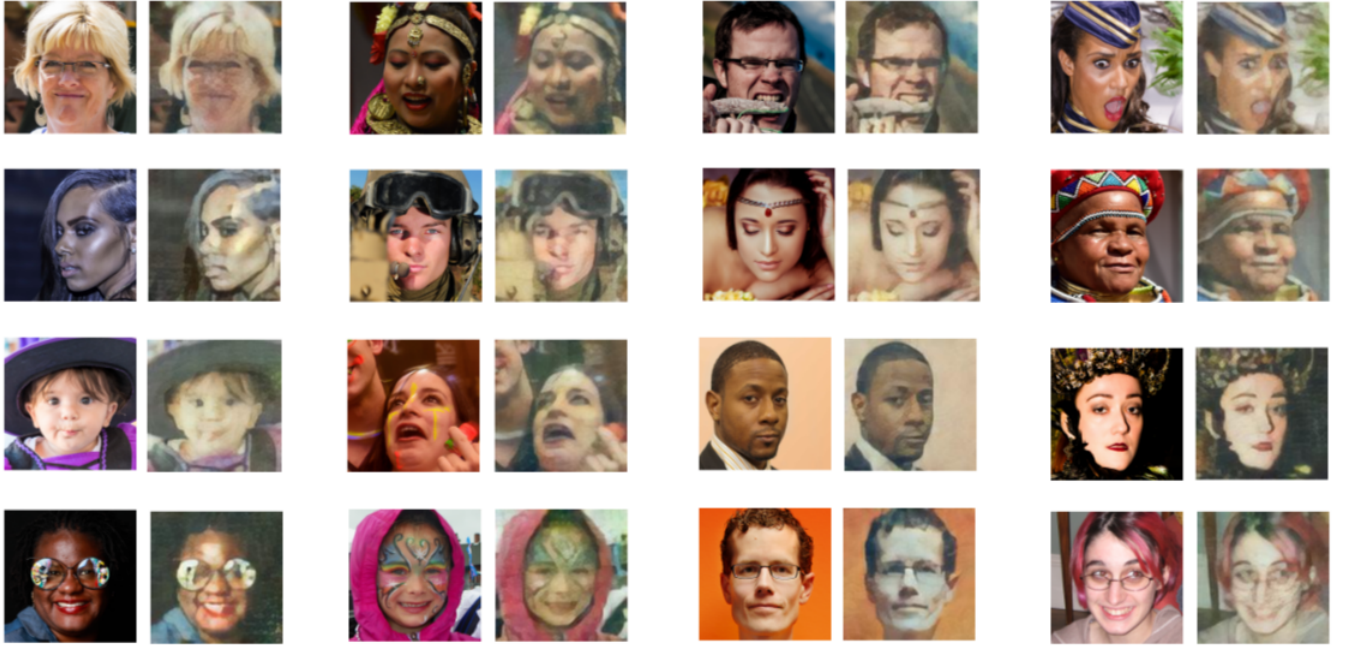
Style transferring with cycleGAN results are presented below. We train the model with 300 epochs. This graph shows the loss used for model training.

Figure 8: Cycle GAN Generator and Discriminator Loss During Training (by epoch)



After training we get the following output (figure 9). As in previous task, pretrained image super resolution model is applied for a better quality.

Figure 9: Examples of Style transferred images



As a result, we conclude that our aim to examine the GAN deep learning models approach and apply it with a real picture images is successfully achieved. As for the first sub-task we applied a more classical model DCGAN, and for the second sub-task a state-of-the-art cycleGAN.

## 6. Future directions

We assume that achieved result is not high-quality level. So for further development we plan: to increase image sizes from  $3 \times 64 \times 64$  to  $3 \times 256 \times 256$  at least, train on bigger data, increase number of epochs.

Also, we want to apply idea from the paper “Resolution Dependent GAN Interpolation for Controllable Image Synthesis Between Domains” what seems an interesting topic to investigate. Interpolation between a base model trained on FFHQ and a transferred model trained on ukiyo-e faces. Resolution dependent model interpolation creates new models which generate images from a novel domain. Depending on which model the high and low resolution layers are taken from, we can select either structural characteristics of ukiyo-e faces with photo-realistic rendering or vice-versa. Furthermore, by selecting which layers are swapped we can tune the effect.

## References

- [1] Ian Goodfellow. *Generative Adversarial Networks*. 2014. URL: <https://arxiv.org/abs/1406.2661>.

- [2] Ian Goodfellow. *Tutorial: Generative Adversarial Networks*. 2016. URL: <https://arxiv.org/abs/1701.00160>.
- [3] Justin Pinkney. *Ukiyo-e faces dataset*. 2020. URL: <https://www.justinpinkney.com/ukiyoe-dataset/>.
- [4] Alec Radford. *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*. 2015. URL: <https://arxiv.org/abs/1511.06434>.
- [5] Jun-Yan Zhu. *Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks*. 2017. URL: <https://arxiv.org/abs/1703.10593>.